VoxelViz: Progressive Voxel-Based Visualization of 3D Models with Animated Build-Up

Vincent Jovian Christanto
National Tsing Hua University
Taiwan

Aryabima Mandala Putra
National Tsing Hua University
Taiwan

Abstract - VoxelViz is a proposed visualization tool that uses a progressive voxel-based representation of 3D models combined with animated build-up sequences to aid education and creativity rules. This survey reviews two key areas of related research: voxel-based visualization methods and animated visualization methods. Voxel-based approaches represent 3D objects as volumetric pixels (voxels) and have been shown to efficiently capture complex shapes, support volumetric analyses, and enable interactive rendering even for billion-voxel models. Animated visualization techniques, meanwhile, leverage time-based transitions to gradually reveal structure, which studies have found can significantly improve user engagement, comprehension, and spatial understanding compared to static visuals. We summarize representative research in each category, discussing their methodologies, purposes, and contributions. We then examine the feasibility of these methods in real-world implementations and their applications in domains such as education, engineering, and science. Finally, we compare voxel-based visualization with traditional mesh and point-based techniques, highlighting that the combination of voxel representations with animated progressive reveal can provide unique benefits for learning and scientific insight by making complex 3D information more accessible, intuitive, and interactive.

1. INTRODUCTION

In recent years, voxel-based graphics have become increasingly popular in fields ranging from gaming and simulation to education and medical imaging. Voxels—short for volumetric pixels—represent 3D space as a grid of equally sized cubes, offering a structured way to capture both surface and internal features of models. Their discrete, grid-aligned format makes them ideal for algorithms involving collision detection, slicing, volumetric data analysis, and interactive editing, which are often cumbersome in traditional mesh representations.

The motivation for this survey stems from the development of *VoxelViz*, a proposed real-time educational tool that converts 3D models into voxel form and animates their construction progressively. This visual "build-up" effect, reminiscent of stacking LEGO bricks, provides learners with a more tangible understanding of complex shapes and spatial structures. By animating how a 3D form is assembled voxel by voxel, *VoxelViz* aims to enhance engagement and comprehension, particularly for students in STEM and design-related fields.

To assess the feasibility of implementing *VoxelViz*, this paper surveys recent research and real-world implementations across two domains:

- **Voxelization Techniques of 3D Models :** How to efficiently convert 3D triangle meshes into voxel grids, including surface and solid voxelization, coverage strategies, and performance trade-offs.
- **Voxel-Based Animation and Visualization Techniques :** How to render, animate, and interact with voxel models in real-time, particularly in educational contexts.

We focus on methods that support real-time applications, can handle textured models, and are compatible with current GPU hardware. The goal is not only to analyze the technical viability of *VoxelViz* but also to highlight the educational value of voxel-based representations in comparison to mesh or point cloud approaches.

2. SURVEY

2.1 Voxelization Techniques of 3D Models

This section surveys methods for creating voxel representations from meshes, including surface vs. solid voxelization, handling of sparse or textured voxels, key properties like *coverage* (cover, supercover, partial cover) and *connectivity*, common algorithms (scanline, voxel intersection tests, GPU rasterization), and evaluations of accuracy and performance in real-time contexts.

2.1.1 Voxel Types and the VoxelViz Context

Voxelization methods generally fall into two primary categories: **surface voxelization and solid voxelization**. Surface voxelization represents only the boundary of the model, resulting in fewer voxels and reduced computational demand, ideal for visualization purposes. In contrast, solid voxelization includes the entire internal volume, beneficial for simulations and structural analyses but at increased memory and processing costs, necessitating compression techniques for

efficiency. For our project VoxelViz, which aims to animate the build-up of a 3D model progressively from bottom to top (and potentially in other orders), solid voxelization is the most appropriate. This is because surface voxelization may result in hollow structures, which break the illusion of a model forming realistically. Solid voxelization provides a filled-in, block-by-block construction experience that better supports the pedagogical goal.

Coverage refers to how voxels represent the surface geometry. Methods include cover (minimal boundary representation), supercover (maximum boundary representation ensuring no gaps), and partial cover (partial voxel occupancy). Supercover methods, while preventing gaps and ensuring continuity, can produce thicker surface appearances due to voxel overlap. Partial cover methods offer thinner, more visually accurate approximations. Connectivity, critical for model integrity, includes 6-connected (face-connected voxels ensuring watertightness), 18-connected, and 26-connected approaches, each affecting visual and structural coherence. Cohen-Or and Kaufman's criteria (accuracy, separability, and minimality) are commonly applied to evaluate voxelization quality, emphasizing avoiding unintended connections and ensuring accurate shape approximation.

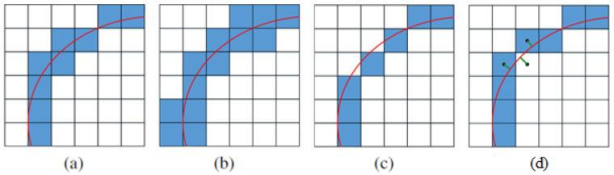

Fig. 1. Representation of a cover (a), supercover (b), partial cover (c), and partial cover (d) well-voxelized curve in 2D.

2.1.2 Voxelization Algorithm

Several primary algorithmic approaches include scanline, voxel intersection tests, and GPU-based rasterization:

Scanline Methods : Early voxelization techniques adapted 2D scan-conversion methods. Scanline algorithms traverse model slices to identify mesh intersections, filling intermediate voxels for solid voxelization or marking boundaries for surface voxelization.

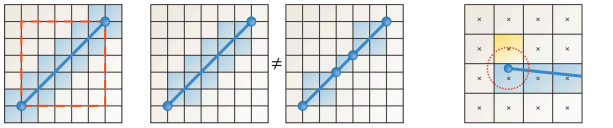

Fig. 2. The voxelization should be independent of planar surface tessellations. (a) Hence, touched voxels outside the bounding box must also be considered. (b) But the simple distance criteria by *Huang et al. [1998]* may wrongly include additional voxels.

Amanatides and Woo's voxel traversal and Bresenham algorithms effectively voxelize lines, providing the foundation for triangle voxelization methods. *Håkansson (2020)* highlighted floating-point "Real Line Voxelization" (RLV) for superior accuracy and performance over integer methods.

In the following figures the yellow voxels are voxels in both algorithms. The red voxels are only in the algorithm mentioned first and the blue voxels are only in the algorithm mentioned last.

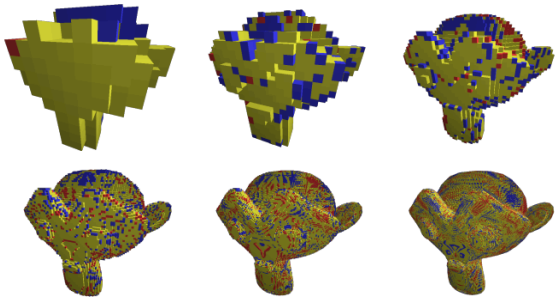

Fig. 3. : Difference between RLV and Bresenham for the Blender monkey at different resolutions.

Zhang et al. (2017) further optimized scanline voxelization by integer-based slicing, significantly improving speed on both CPU and GPU platforms.

Intersection-Based Methods : Intersection tests examine each voxel for overlaps with model triangles, a direct but computationally intensive method optimized through GPU-accelerated conservative rasterization. *Schwarz and Seidel (2010)* implemented GPU-based triangle-box overlap tests,

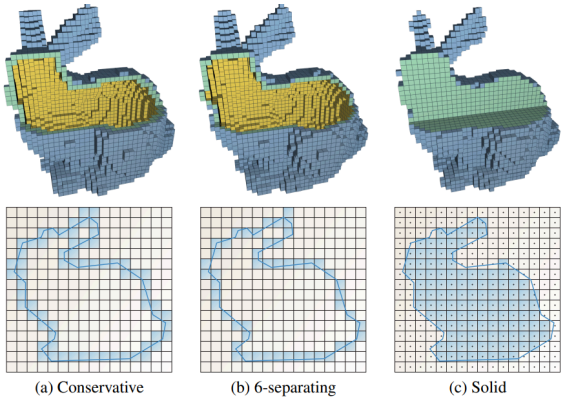

Fig. 4. : Different kinds of voxelizations covered.

significantly enhancing voxelization speed and enabling conservative coverage guarantees, making it suitable for interactive applications.

GPU Methods and Hybrid Approaches : Leveraging GPU parallelism, *Schwarz and Seidel (2010)* introduced direct octree voxelization, reducing memory usage by sparsely populating the voxel grid.

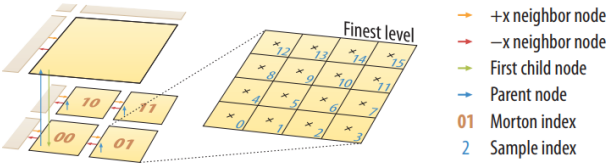

Fig. 5. : Overview of the octree structure (2D analog).

Hybrid methods, such as incremental voxelization by *Zhou et al. (2017)*, combine intersection and scanline techniques for efficiency in dynamic environments. Modern GPU implementations, including NVIDIA's VoxelPipe and Gigavoxels, demonstrate the feasibility of real-time voxelization in interactive contexts, managing complex scenes efficiently through CUDA parallelization and spatial data structures.

2.1.3 Accuracy, Resolution, and Performance Evaluation

Voxelization accuracy is inherently tied to grid resolution; higher resolutions yield more detailed representations but demand increased memory and processing power. Methods differ in handling sub-voxel details, ranging from binary occupancy to fractional coverage for anti-aliasing. Conservative methods are particularly effective in capturing thin features without omissions. Adaptive sampling techniques offer potential efficiency gains by varying resolution according to model complexity.

Performance evaluations highlight substantial improvements in voxelization speeds. On contemporary GPUs, voxelizing models with hundreds of thousands of triangles into high-resolution grids (e.g., 512^3) can be completed within milliseconds, making real-time applications feasible. Techniques such as spatial hashing and sparse representations further optimize performance by minimizing redundant computations and memory usage.

Method	How It Works	Pros	Cons
Scanline Voxelization *Zhang et al. (2017), Håkansson (2020)*	Slice mesh along one axis and use 2D line rasterization to mark voxel spans	Fast, accurate for surface; integer operations; GPU-friendly	Only surface voxelization; needs triangulation cleanup
Triangle-Box Overlap Test *Schwarz & Seidel (2010)*	For each triangle, check which voxels it overlaps using bounding-box intersection	Very accurate, produces watertight voxels, supports solid/surface	Slight over voxelization (supercover), requires more math
GPU Conservative Rasterization *Schwarz & Seidel (2010)*	Render triangles in 3 orthogonal views; voxels touched by any projection are marked	Real-time GPU acceleration; highly parallel	Conservative → more voxels than needed
Octree-based Sparse Voxelization *Schwarz & Seidel, OpenVDB*	Use octree to store only occupied voxels; subdivide space adaptively	Saves memory; good for large models	Complex to implement; slower for small objects
Bresenham Line-Based Filling *Håkansson (2020)*	Use integer voxel traversal (3D Bresenham) to fill triangles	Lightweight and efficient	Integer rounding errors, lower precision

Among the surveyed methods, the **triangle-box overlap technique by Schwarz and Seidel (2010)** stands out as the most effective for real-time applications like *VoxelViz*. It supports both surface and solid voxelization, produces watertight and 6-connected outputs, and runs efficiently on modern GPUs. While alternatives like scanline methods are simpler and fast, they lack the same coverage guarantees. Sparse voxel structures such as octrees or OpenVDB are powerful for large-scale data but are more complex and less suitable for interactive educational tools. For *VoxelViz*, which prioritizes speed, clarity, and animation, the triangle-box method offers the best balance of performance and quality.

2.1.4 Sparse and Textured Voxels

Sparse voxel structures (octrees, sparse matrices, hash grids) efficiently represent voxel models by storing only occupied or relevant regions, crucial for managing memory in large-scale scenes. Textured voxelization assigns voxel attributes based on original mesh textures, significantly enhancing visual fidelity for educational models. Frameworks like NanoVDB provide robust support for dynamic voxel attributes, enabling realistic and interactive visualization scenarios.

In summary, voxelization techniques today are highly efficient and accurate, capable of real-time generation of detailed voxel models. These advancements enable dynamic educational and scientific visualizations, enhancing both interactivity and comprehensibility in complex spatial contexts.

2.2 Voxel-Based Animation and Visualization Techniques

Voxel-based animation and visualization techniques leverage voxel representations to create dynamic, intuitive, and educationally effective visual experiences. Unlike static representations, animated visualizations progressively reveal structural details, significantly enhancing spatial comprehension and user engagement. This section explores various animation methods and real-time implementations, highlighting key algorithms, their advantages, and practical applications, specifically targeting real-time educational tools such as VoxelViz.

2.2.1 Progressive Voxel Animations

Progressive voxel animations systematically construct 3D models by sequentially revealing or assembling individual voxels. Such animations enhance understanding by visually demonstrating the build-up of complex structures. Hedayati et al. (2019) introduced robust sequencing algorithms in their "Material-Robot System for Discrete Cellular Assembly,"

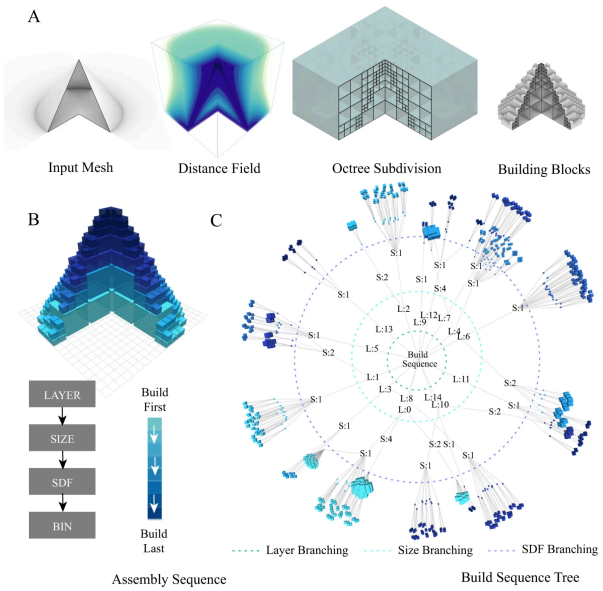

Fig. 6. : The hierarchical building blocks colored according to the assembly sequence.

where voxel sequences follow stability-driven strategies like layer-by-layer and wavefront approaches. Their method orders voxel insertions to ensure structural integrity, effectively translating to educational animations that visually depict incremental construction processes. Integrating Hedayati et al.'s discrete sequences, often represented as JSON arrays of voxel coordinates. VoxelViz can render structured frame-by-frame build-ups, providing clear and intuitive visualizations.

Dai et al. (2018) presented a novel method for fabricating 3D models on a robotic printing system equipped with multi-axis motion.

Fig. 7. : 3D printing of solid models.

Their approach accumulates materials inside the volume along curved tool-paths, significantly reducing or even eliminating the need for supporting structures. This strategy involves two successive decompositions: volume-to-surfaces and surfaces-to-curves. The volume-to-surfaces decomposition is achieved by optimizing a scalar field within the volume that represents the fabrication sequence. The field is constrained such that its iso-values represent curved layers supported from below, presenting a convex surface that allows for collision-free navigation of the printer head. After extracting all curved layers, the surfaces-to-curves decomposition covers them with tool-paths while considering constraints from the robotic printing system. This method successfully generates tool-paths for 3D printing models with large overhangs and high-genus topology. VoxelViz can specifically benefit from Dai et al.'s methodology by adapting the multi-stage decomposition approach, allowing for effective representation and animation of complex models with intricate features or overhangs. While originally designed for physical 3D printing, the principle of sequentially decomposing complex geometries into manageable layers directly supports VoxelViz's goal of clear visualization.

2.2.2 GPU-Accelerated Real-Time Visualization

Smooth, real-time voxel animations rely significantly on GPU acceleration. Schwarz and Seidel developed GPU-based conservative voxelization methods, marking voxels intersected by triangle projections across multiple orthogonal views, ensuring watertight and connected voxel grids. Leveraging CUDA-based parallelization, their technique provides rapid rendering, essential for real-time interactivity in educational contexts like VoxelViz. GPU frameworks such as NVIDIA's VoxelPipe and Gigavoxels further build on similar principles, using sparse data structures to efficiently render billions of voxels interactively. These frameworks inform VoxelViz's GPU-accelerated voxel rendering, enabling real-time animations and interactive visualizations.

2.2.3 Algorithmic Approaches to Voxel Animation

Several animation algorithms control voxel appearance and motion:

- Layer-by-Layer Animation: Pandey et al. (2006) illustrated layered voxel animations that sequentially visualize cross-sections, effectively mimicking real-world manufacturing processes. VoxelViz can utilize these algorithms to visually represent structural assembly, highlighting spatial relationships and internal structures clearly and progressively.

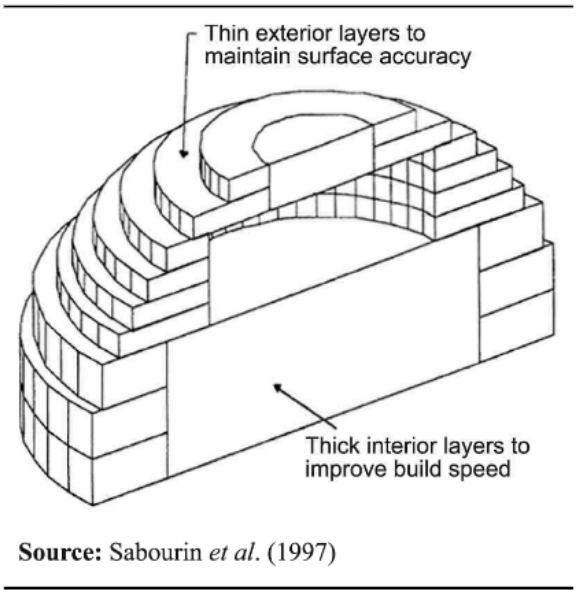

Fig. 8. : Concept of accurate interior and exterior slicing.

- Wavefront and Spiral Animations: wavefront voxel sequencing algorithms that progressively animate voxels expanding outward from a seed or boundary

voxel, emphasizing structural connectivity. Spiral animations, as discussed by Crassin et al. (2012), progressively reveal structures from central points outward, providing engaging and intuitive visual effects suitable for educational applications.

- Octree-Based Hierarchical Animation: Schwarz and Seidel (2010) implemented hierarchical animation techniques through octree voxelization, progressively refining voxel resolution from coarse to fine. This hierarchical approach introduces complexity incrementally, helping manage cognitive load by gradually presenting structural details, a valuable feature for educational purposes.

- Adaptive and Context-Aware Sequences: Aleksandrov et al. (2021) explored voxel clustering methods for animation sequences, prioritizing regions based on feature saliency or structural importance. Such data-driven sequencing can enhance educational visualizations by highlighting critical model features early in the animation process.

2.2.4 Implementation Considerations for Animated Voxel Visualizations

Implementing voxel animations involves three primary stages:

1. Voxelization: Convert 3D meshes into voxel grids using efficient voxelization techniques, typically employing GPU-based rasterization methods for performance optimization.

2. Sequencing: Determine voxel appearance order using strategies such as layer-by-layer, wavefront, or spiral sequences. These sequencing strategies facilitate clear visual progression, enhancing educational value.

3. Animation and Rendering: Implement voxel animations through GPU acceleration using visualization libraries such as Trimesh, PyVista, or Blender. Consistent with the implementation described by the Medium platform, voxel sequences are progressively visualized, starting from foundational layers or key structural points and advancing towards complete model formation. Intermediate frames can be generated systematically and compiled into animations, providing a smooth visual narrative suitable for educational purposes.

After sequencing voxel visibility, individual frames can be rendered and compiled into animations using software like Blender or image processing libraries, providing smooth and intuitive visual narratives.

2.2.5 Feasibility and Educational Value

Real-world applications across engineering education, medical training, and scientific visualization confirm the feasibility and educational efficacy of animated voxel representations. Studies by Hedayati et al. (2019), Tümer and Kara (2013), Li et al. (2014), and Schwarz and Seidel (2010) highlight the substantial advantages of progressive voxel-based animations in enhancing user engagement and spatial comprehension. The proposed VoxelViz tool leverages these insights, combining voxel representations with carefully designed animated sequences to maximize educational impact, making complex 3D structures both accessible and comprehensible to learners.

3. DISCUSSION & CONCLUSION

The survey of voxelization and animation techniques reveals that a system like *VoxelViz* is not only feasible but also well-aligned with the direction of current research and practice. Advances in both CPU and GPU voxelization algorithms—such as projected scanline methods and conservative rasterization—allow fast and accurate conversion of even complex 3D meshes into voxel representations. These methods can be implemented in real time, supporting interactive applications that require immediate feedback as users upload or modify models.

The animation component, while less standardized in academic literature, can be effectively implemented using progressive reveal strategies, slicing planes, or procedural fill patterns. Educational visualization does not necessarily demand deformable animations or ultra-high fidelity, making simpler techniques like bottom-up build-up or spiral construction highly suitable. These methods are not only computationally lightweight but also intuitive for learners.

Voxel-based visualization offers several clear advantages for education: it allows learners to "see inside" models, supports robust manipulation like adding/removing voxels, and handles complex shapes without the topological issues that often arise in mesh-based tools. Compared to point clouds, voxels provide structured spatial relationships, which are beneficial for building animations and interactions. Compared to polygon meshes, voxels offer a more approachable and editable format—especially in educational contexts where the focus is on conceptual understanding rather than visual realism.

However, trade-offs remain. Voxels require careful management of resolution versus performance: too coarse and the model loses detail; too fine and real-time interaction becomes difficult. Memory consumption can also be high, especially for solid voxelizations or when storing texture data per voxel. Techniques like sparse voxel octrees and GPU instancing help mitigate these concerns but require additional complexity.

In summary, the core components of *VoxelViz*—real-time voxelization, texture-aware voxel coloring, and progressive animation—are supported by state-of-the-art research and practical implementations. By leveraging these, *VoxelViz* can

become a powerful educational tool that not only visualizes 3D models but also teaches spatial reasoning through interactive exploration. The next step would involve integrating these technologies into an intuitive user interface and evaluating their effectiveness in real classroom settings.

4. CONTRIBUTIONS

(Category 1) Voxelization Techniques of 3D Models (2.1) : Vincent Jovian Christanto

(Category 2) Voxel-Based Animation and Visualization Techniques (2.2): Aryabima Mandala Putra

REFERENCES

[1] M. Schwarz and H.-P. Seidel, "Fast Parallel Surface and Solid Voxelization on GPUs." ACM SIGGRAPH Asia, 2010. https://michael-schwarz.com/research/publ/files/vox-siga10.pdf#:~:text=voxel%20grids%2C%20we%20introduce%20a,level%20voxels.%20This%20representation

[2] Y. Zhang *et al.*, "Efficient Voxelization Using Projected Optimal Scanline." *Graphical Models*, vol. 94, 2017. https://tianjiashao.com/Docs/2017/voxelization.pdf#:~:text=3D%20geometrically%20complex%20models,scanline%20interval%20can%20be%20obtained

[3] T. Håkansson, *"A Comparison of Optimal Scanline Voxelization Algorithms,"* MSc Thesis, Linköping Univ., 2020. https://liu.diva-portal.org/smash/get/diva2:1459330/FULLTEXT01.pdf

[4] M. Aleksandrov *et al.*, "Voxelisation Algorithms and Data Structures: A Review." *Sensors*, 21(24):8241, 2021. https://pmc.ncbi.nlm.nih.gov/articles/PMC8707769/?

[5] C. Crassin *et al.*, "GigaVoxels: A Voxel-Based Rendering Pipeline for Efficient Exploration of Large Scenes." *IEEE Trans. Vis. Comput. Graphics*, 18(9), 2012. https://inria.hal.science/hal-00840637/file/DanielRiosFinalDraft2.pdf#:~:text=al.%2C%202009,visibility%20informations%20provided%20directly%20by

[6] R. Hedayati, Z. Hu, J. Ranganathan, N. Verma, S. Kim, H. Choset, and M. Sitti, "Material‑Robot System for Discrete Cellular Assembly," MIT CSAIL, 2019. https://www.nature.com/articles/s44172-022-00034-3

[7] C. Dai, C. C. L. Wang, C. Wu, S. Lefebvre, G. Fang, and Y.-J. Liu, "Support-Free Volume Printing by Multi-Axis Motion," *ACM Transactions on Graphics*, vol. 37, no. 4, Article 1, Aug. 2018. https://www.researchgate.net/publication/326729804_Support-free_volume_printing_by_multi-axis_motion

[8] P. M. Pandey, N. V. Reddy, and S. G. Dhande, "Slicing Procedures in Layered Manufacturing: A Review," *Rapid Prototyping Journal*, vol. 12, no. 5, pp. 287–307, 2006. https://www.researchgate.net/publication/241441853_Slicing_procedures_in_layered_manufacturing_A_review